



Lecture 10: Defaults and Rest Props

INTERVIEW QUESTION

[> CORE](#)

How do you give a prop a default value and gather the rest?

Some props are optional, and sometimes you want to forward whatever the parent passed straight to a DOM element. How do defaults and "the rest of the props" work?

When building robust components, you often want to provide sensible defaults for optional settings, while still allowing the parent to customize them. Furthermore, you need a way to gracefully handle native HTML attributes without manually declaring every single one.

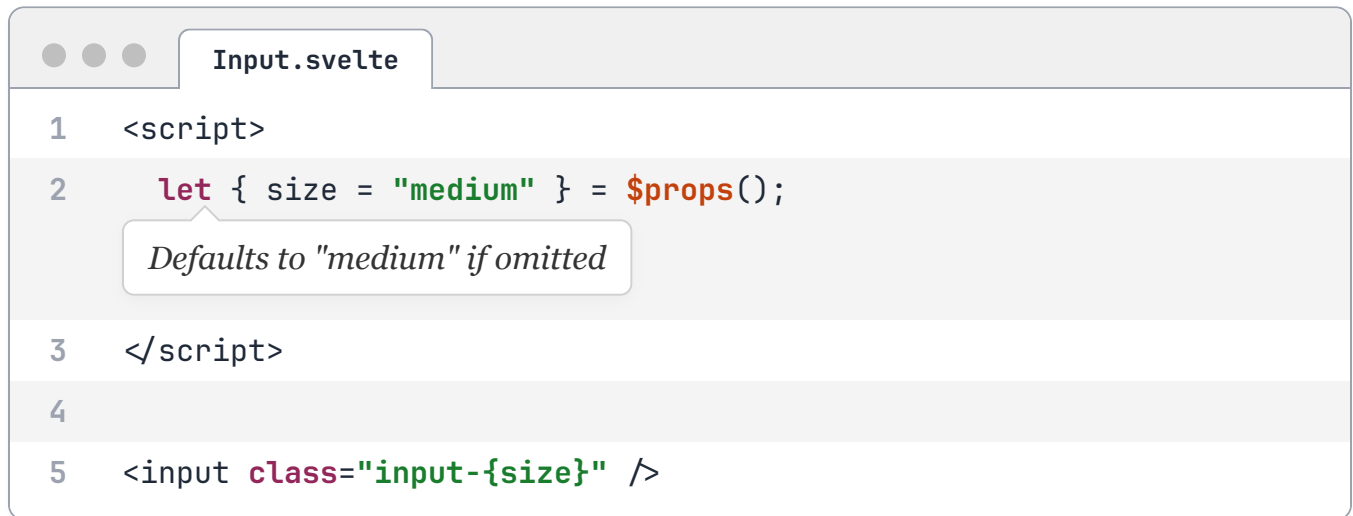
The Exhaustive Declaration Problem

Imagine you are building an `Input.svelte` component. You decide to offer a `size` prop to control its visual scale, but you want it to default to "medium" so developers don't have to specify it every time.

More problematically, a standard HTML `<input>` tag supports dozens of attributes: `disabled`, `readonly`, `placeholder`, `maxlength`, `pattern`, `type`, and more. If you want your custom component to support all of these standard features, manually defining every single one inside your `$props()` block is a massive waste of time. If you forget to declare `pattern`, your component will silently swallow the attribute, breaking form validation and infuriating your teammates who expect your `<Input>` to behave like a native browser element.

Default Values

Because Svelte 5's `$props` rune uses standard JavaScript destructuring syntax, you can assign default fallback values directly in the declaration. If the parent component does not provide the prop, Svelte falls back to the default value automatically.



```
1 <script>
2   let { size = "medium" } = $props();
3 </script>
4
5 <input class="input-{size}" />
```

Now, the parent can render `<Input />` for a standard input, or `<Input size="large" />` for an oversized one.

The Rest Props Pattern

To solve the issue of forwarding native HTML attributes, you can use the JavaScript **rest property** syntax (`...`). By putting `...rest` at the end of your destructuring block, Svelte gathers all the props that you *didn't* explicitly name and bundles them into a single object.

You can then use Svelte's spread syntax (`{...rest}`) in your markup to apply all of those gathered attributes directly onto a DOM element in one fell swoop.

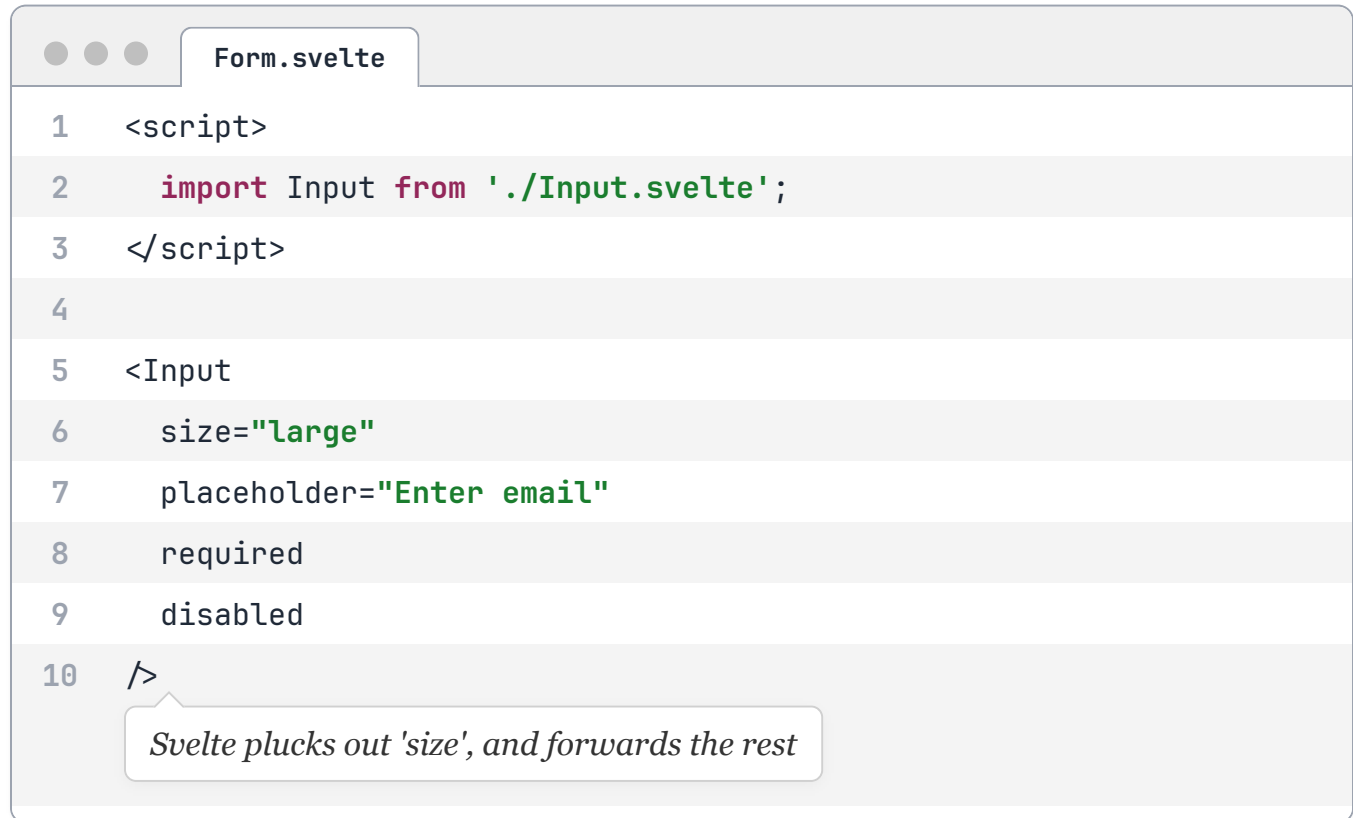
```
1 <script>
2   let {
3     size = "medium",
4     ...rest
5   } = $props();
6 </script>
7
8 <input class="input-{size}" {...rest} />
```

Gathers everything else into an object named 'rest'

Spreads all undeclared attributes onto the native input

Forwarding in Action

When a parent uses this component, they can pass arbitrary HTML attributes without the child component needing to know about them explicitly:



```
1  <script>
2    import Input from './Input.svelte';
3  </script>
4
5  <Input
6    size="large"
7    placeholder="Enter email"
8    required
9    disabled
10 />
```

Svelte plucks out 'size', and forwards the rest

In this scenario, `size` is extracted and used for the CSS class. The `placeholder`, `required`, and `disabled` props are scooped up by the `...rest` object and seamlessly applied directly to the underlying native `<input>` tag.

INTERVIEW TIP

To impress the interviewer: Note that the order of the spread attribute matters. If you write `<input {...rest} class="my-class">`, your static class will overwrite any `class` passed in via `...rest`. Conversely, writing `<input class="my-class" {...rest}>` allows the parent component to override your internal classes.

Summary

Svelte 5 leans entirely on standard JavaScript language features to handle complex prop scenarios. You can set optional fallback values using standard destructuring assignment (`= "default"`). To build flexible wrapper components that mimic native HTML elements, you use the rest operator (`...rest`) to gather all undeclared props, and then use the spread operator (`{...rest}`) in your markup to forward those attributes directly to the underlying DOM node.

	DEFAULTS	REST PROPS
Syntax	<pre>let { size = 'md' } = \$props()</pre>	<pre>let { size, ...rest } = \$props()</pre>
Markup Usage	<pre>{size}</pre>	<pre><input {...rest} /></pre>
Primary Use Case	Optional component configurations	Forwarding native HTML attributes